

Problem Statement or Requirement:

A client's requirement is, he wants to predict the insurance charges based on the several parameters. The Client has provided the dataset of the same.

As a data scientist, you must develop a model which will predict the insurance charges.

1.) Identify your problem statement

Straightword - prediction of insurance charges based on certain parameters

2.) Tell basic info about the dataset (Total number of rows, columns)

Dataset contains:

input_label = "age,sex,bmi,children,smoker"

output_label = "charges"

Total rows = 1338

Total columns = 6

3.) Mention the pre-processing method if you're doing any (like converting string to number – nominal data)

The dataset contains categorical columns (sex, smoker) => encode to nominal data

4.) Develop a good model with r2_score. You can use any machine learning algorithm; you can create many models. Finally, you have to come up with the final model.

I'm listing the r2_score for all the model:

Linear = 0.7680881643600721

SVM = 0.8506525310780688

Decision Tree = 0.8628846625378732

Random Forest = 0.8603594648119404

AdaBoosting= 0.8260955268362907

GradientBoost = 0.8695606981049682

XGBoost = 0.8703282507178823 --> Higher

XGradient Boosting has the better score

5.) Mention your final model, justify why u have chosen the same

Based on the manual testing and evaluating using GridSearchCv 'XGBoost' performed better and efficiently than other models.

SVM Predicted Output:

s.no	kernel	C	gamma	Prediction
1	linear	1	0.1	-0.087320991
2	linear	1	1	-0.087320991
3	linear	1	3	-0.087320991
4	linear	10	0.1	0.126687376
5	linear	10	1	0.126687376
6	linear	10	3	0.126687376
7	linear	100	0.1	0.594031093
8	linear	100	1	0.594031093
9	linear	100	3	0.594031093
10	linear	1000	3	0.743111829
11	rbf	1	0.1	-0.082699538
12	rbf	1	1	-0.083056123
13	rbf	1	3	-0.083077463
14	rbf	10	0.1	-0.081209443
15	rbf	10	1	-0.082797322
16	rbf	10	3	-0.083010521
17	rbf	100	0.1	-0.080299947
18	rbf	100	1	-0.080648331
19	rbf	100	3	-0.082008068
20	rbf	1000	3	-0.07855458
21	rbf	1000	3	-0.07855458
22	poly	1000	3	0.594031093
23	sigmoid	1	0.1	-0.083084907
24	sigmoid	1	1	-0.083084907
25	sigmoid	1	3	-0.083084907
26	sigmoid	10	0.1	-0.083084907
27	sigmoid	10	1	-0.083084907
28	sigmoid	10	3	-0.083084907
29	sigmoid	100	0.1	-0.083084907
30	sigmoid	100	1	-0.083084907
31	sigmoid	100	3	-0.083084907

Decision Tree Prediction:

s.no	criterion	splitter	max_depth	ndom_sta	prediction
1	squared_error	best	5	42	0.857593287
2	squared_error	best	10	42	0.779209867
3	squared_error	best	15	42	0.73744275
4	squared_error	best	20	42	0.733166753
5	squared_error	best	None	42	0.733166753
6	squared_error	random	5	42	0.829007309
7	squared_error	random	10	42	0.781365433
8	squared_error	random	15	42	0.731983383
9	squared_error	random	20	42	0.746041037
10	absolute_error	best	5	42	0.862884663
11	absolute_error	best	10	42	0.784266619
12	absolute_error	best	15	42	0.779185458
13	absolute_error	best	20	42	0.764307327
14	absolute_error	random	5	42	0.780085037
15	absolute_error	random	10	42	0.799444355
16	absolute_error	random	15	42	0.74015092
17	absolute_error	random	20	42	0.725704071
18	friedman_mse	best	5	42	0.857593287
19	friedman_mse	best	10	42	0.776649904
20	friedman_mse	best	15	42	0.757730505
21	friedman_mse	best	20	42	0.754185775
22	friedman_mse	random	5	42	0.829007309
23	friedman_mse	random	10	42	0.828930496
24	friedman_mse	random	15	42	0.693877386
25	friedman_mse	random	20	42	0.672612909
26	poisson	best	5	42	0.857593287
27	poisson	best	10	42	0.770189336
28	poisson	best	15	42	0.753807767
29	poisson	best	20	42	0.749474474
30	poisson	random	5	42	0.828945878
31	poisson	random	10	42	0.808622291
32	poisson	random	15	42	0.747720187
33	poisson	random	20	42	0.712583615
34	poisson	random	20	42	0.712583615
35	poisson	random	20	42	0.712583615

Random Forest Prediction:

s.no	criterion	n_estimators	max_depth	max_features	random_state	prediction
1	squared_error	50	10	sqrt	0	0.857454225
2	squared_error	50	10	log2	0	0.857454225
3	squared_error	50	10	None	0	0.845938936
4	squared_error	50	20	sqrt	0	0.853603238
5	squared_error	50	20	log2	0	0.853603238
6	squared_error	50	20	None	0	0.842871427
7	squared_error	50	None	sqrt	0	0.853502498
8	squared_error	50	None	log2	0	0.853502498
9	squared_error	50	None	None	0	0.842892006
10	squared_error	100	10	sqrt	0	0.86325739
11	absolute_error	50	10	sqrt	0	0.862515345
12	absolute_error	50	10	log2	0	0.862515345
13	absolute_error	50	10	None	0	0.849951825
14	absolute_error	50	20	sqrt	0	0.850755205
15	absolute_error	50	20	log2	0	0.850755205
16	absolute_error	50	20	None	0	0.836977115
17	absolute_error	50	None	None	0	0.837242422
18	friedman_mse	50	10	sqrt	0	0.857720884
19	friedman_mse	50	10	log2	0	0.857720884
20	friedman_mse	50	10	None	0	0.845327098
21	friedman_mse	50	20	sqrt	0	0.850755205
22	friedman_mse	50	20	log2	0	0.850755205
23	friedman_mse	50	20	None	0	0.836977115
24	friedman_mse	50	None	None	0	0.837242422
25	poisson	50	10	sqrt	0	0.859323711
26	poisson	50	10	log2	0	0.859323711
27	poisson	50	10	None	0	0.848288721
28	poisson	50	20	sqrt	0	0.852030865
29	poisson	50	20	log2	0	0.852030865
30	poisson	50	20	None	0	0.844586316
31	poisson	50	None	sqrt	0	0.852432297
32	poisson	50	None	log2	0	0.852432297
33	poisson	50	None	None	0	0.844528208